
Université Abdelmalek Essaâdi

Faculté des Sciences et Techniques – Tanger

Département Informatique

Licence : Analytique des Données

Miniprojet 2

Data Lake & Analyse du Marché de l'Emploi IT au Maroc

Mexora RH Intelligence

Réalisé par :

Ouissal SEKKARI

Abir FATAH

Encadré par :

Mr. Hassan ZILI

Implémentation du Pipeline ETL

Structure de Répertoires du Data Lake

La figure 1 présente la structure de répertoires générée sur la machine après exécution du script `bronze_ingestion.py`. Elle matérialise l'organisation en zones Bronze / Silver / Gold avec le partitionnement source/mois en Bronze et ville/mois en Silver.

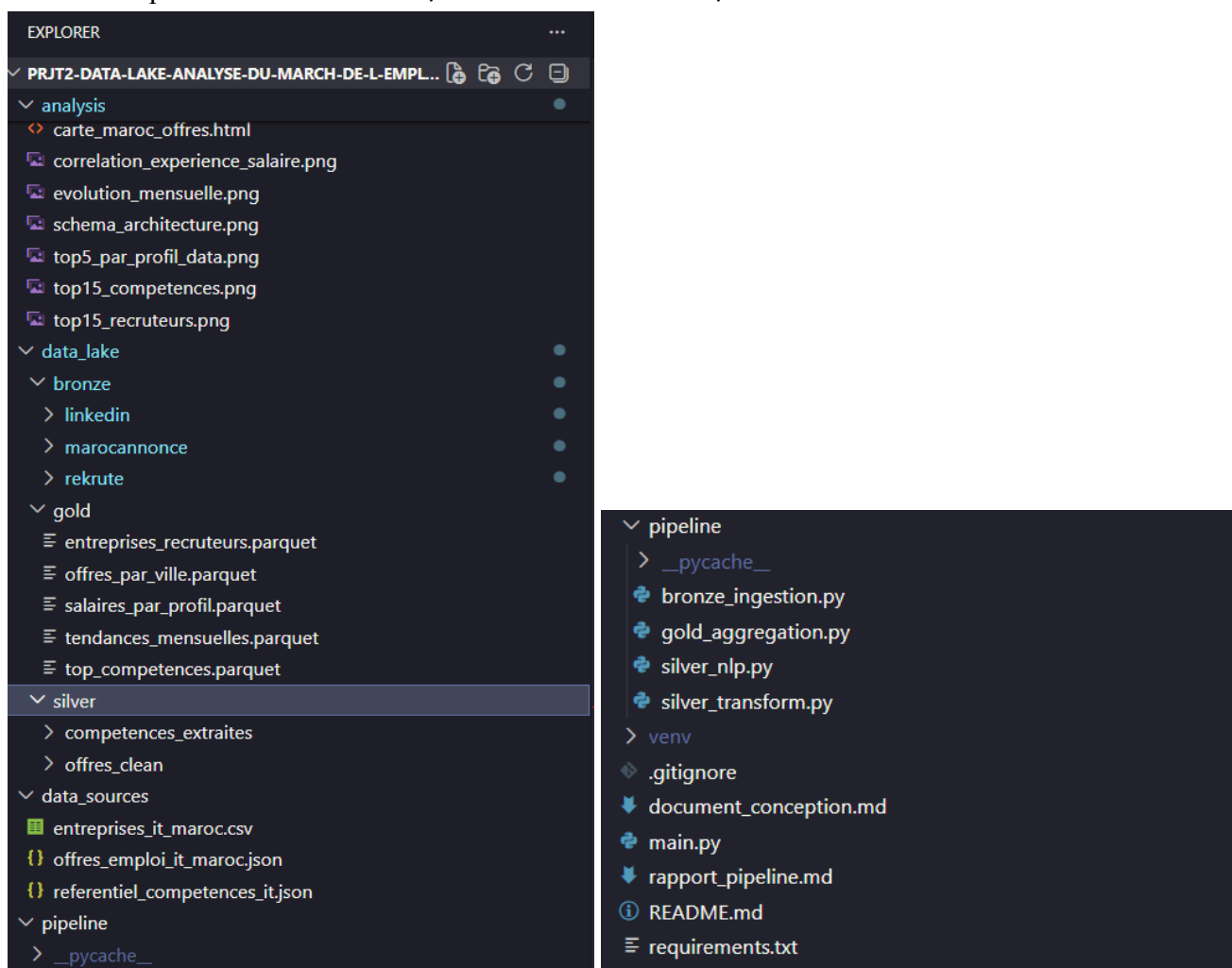


FIGURE 1 – Structure de répertoires du Data Lake Mexora après exécution du pipeline

Rapport d'Exécution du Pipeline

Le rapport ci-dessous documente les quatre étapes principales du pipeline ETL : l'ingestion des données brutes en zone Bronze, leur transformation et standardisation en zone Silver, l'extraction NLP des compétences, ainsi que l'agrégation Gold destinée à produire les tables analytiques.

Ingestion Bronze

TABLE 1 – Métriques globales d'ingestion — Zone Bronze

Métrique	Valeur
Offres ingérées	5 000
Partitions créées	69
Date d'ingestion	2026-05-04T18:51:01

TABLE 2 – Répartition des offres par source — Zone Bronze

Source	Offres
LinkedIn	1 696
MarocAnnonce	1 612
Rekrute	1 692
Total	5 000

Règle appliquée : les données sont copiées telles quelles depuis la source, sans aucune modification. La zone Bronze est *immuable*. Les cas limites — dates exprimées en plusieurs formats (YYYY-MM-DD, DD/MM/YYYY, DD-MM-YYYY) — sont stockés tels quels ; leur normalisation est réservée à la zone Silver.

Transformation Silver — Nettoyage

Normalisation des dates

TABLE 3 – Métriques de normalisation des dates

Métrique	Valeur
Dates valides	5 000
Dates invalides / inconnues	0
Incohérences corrigées (publication > expiration)	253

Trois formats de date ont été détectés et normalisés vers YYYY-MM-DD. Les dates d'expiration antérieures à la date de publication ont été invalidées (valeur mise à NULL).

Normalisation des villes

TABLE 4 – Métriques de normalisation des villes

Métrique	Valeur
Offres avant traitement	5 000
Villes distinctes (après)	11
Offres avec ville inconnue	0

Un mapping *regex* insensible à la casse a été appliqué, complété par la suppression des espaces parasites. Exemple : "CASABLANCA", "casa", "Casablanca " → "Casablanca". La région administrative a été ajoutée depuis un dictionnaire statique.

Standardisation des types de contrats

TABLE 5 – Répartition des offres par type de contrat

Type de contrat	Offres
CDD	1 340
Stage	1 014
CDI	1 003
Freelance	979
Alternance	664
Total	5 000

Des expressions régulières appliquées sur la valeur brute en minuscules ont permis d'unifier les variantes. Exemple : "Contrat à durée indéterminée", "cdi", "Permanent" → "CDI".

Normalisation des salaires

TABLE 6 – Métriques de normalisation des salaires

Métrique	Valeur
Offres avec salaire valide	3 918 (78,4 %)
Offres sans salaire (null/confidentiel)	1 082
Salaire médian global (MAD)	17 500

Les montants numériques ont été extraits par regex. La conversion EUR → MAD a été réalisée au taux de référence 2024 (**1 EUR = 10,8 MAD**). Les mentions "K" ont été converties (×1 000) et les valeurs validées dans la fourchette **3 000 – 100 000 MAD**.

Cas limites traités :

- "Selon profil", "Confidentiel", null → salaire_connu = False
- Montants en EUR convertis en MAD
- "15K-20K" → sal_min = 15 000, sal_max = 20 000

Parsing de l'expérience requise

TABLE 7 – Métriques de parsing de l'expérience

Métrique	Valeur
Expérience parsée	4 708
Non parsée	292

Le parsing s'effectue en deux passes : fourchettes explicites d'abord, puis minimum seul. Les

mots-clés débutant / junior sont mappés sur (0 – 2 ans) et senior / expert sur (5+ ans, non borné).

Normalisation des titres de poste

TABLE 8 – Répartition des offres par profil normalisé

Profil normalisé	Offres
Data Analyst	861
Data Engineer	694
Autre IT	564
Data Scientist	461
Développeur Backend	412
Architecte IT	336
Cybersécurité	321
Chef de Projet IT	235
Développeur Full Stack	205
Développeur Frontend	201
DevOps / SRE	173
Développeur Mobile	119
Cloud Engineer	114
QA / Test Engineer	104
Admin Systèmes & Réseaux	103
Tech Lead	97
Total	5 000

Plus de 20 patterns regex couvrent les principales familles de postes IT. Les offres non classifiées sont conservées sous le label "Autre IT" avec un flag d’audit pour revue manuelle.

Extraction NLP – Compétences Silver

TABLE 9 – Métriques globales d’extraction NLP des compétences

Métrique	Valeur
Lignes compétences générées	35 042
Offres avec ≥ 1 compétence	5 000
Offres sans compétence détectée	0
Compétences uniques détectées	70
Moyenne de compétences par offre	7.0

TABLE 10 – Top 10 des compétences les plus demandées sur le marché IT marocain

Compétence	Famille	Mentions
docker	DevOps	2 176
sql	Langages	1 897
git	DevOps	1 881
agile	Méthodologies	1 771
aws	Cloud	1 769
gcp	Cloud	1 706
javascript	Langages	1 224
dbt	Data Engineering	1 128
azure	Cloud	1 013
kubernetes	DevOps	1 004

Stratégie NLP mise en œuvre :

- **Sources** : champs competences_brut et description concaténés avant analyse.
- **Matching** : expressions régulières avec délimiteurs de mots (\b alias \b).
- **Ordre de priorité** : alias triés par longueur décroissante pour éviter les faux positifs (ex : node.js avant node).
- **Déduplication** : une compétence comptabilisée une seule fois par offre, quel que soit le nombre de mentions.

Cas limites traités :

- "node" vs "node.js" : résolu par tri longueur décroissante.
- Séparateurs variés (/, •, ;) : nettoyés avant matching.
- Offres sans description : tracées avec le statut non_détecté.

Agrégation Gold (DuckDB)

La zone Gold est construite par gold_aggregation.py via DuckDB. Les requêtes SQL sont exécutées directement sur les fichiers Parquet Silver, sans chargement préalable dans une base de données serveur. Cette étape transforme les données nettoyées en tables analytiques prêtes pour le notebook, le dashboard et le rapport final.

Vue d'ensemble

Cinq tables analytiques sont produites au format Parquet Snappy. Elles couvrent les compétences les plus demandées, les statistiques salariales, la répartition géographique des offres, les recruteurs actifs et les tendances mensuelles.

Table 11 – Métriques globales de construction Gold

Table Gold	Lignes produites	Colonnes	Source principale
top_competences.parquet	409	6	competences.parquet
salaires_par_profil.parquet	357	11	offres_clean.parquet
offres_par_ville.parquet	2275	8	offres_clean.parquet
entreprises_recruteurs.parquet	100	8	offres_clean.parquet
tendances_mensuelles.parquet	366	7	offres_clean.parquet

Table Gold : top_competences.parquet

Objectif : classer les compétences les plus demandées par profil IT, avec leur poids relatif dans l'ensemble des offres.

Colonnes produites : profil, famille, competence, nb_offres_mentionnent, pct_offres_total, rang_dans_profil.

Requête DuckDB :

```

DuckDB SQL – top_competences.parquet

SELECT
  profil,
  famille,
  competence,
  COUNT(DISTINCT id_offre) AS nb_offres_mentionnent,
  ROUND(
    COUNT(DISTINCT id_offre) * 100.0 /
    (SELECT COUNT(DISTINCT id_offre) FROM competences),
    2
  ) AS pct_offres_total,
  RANK() OVER (
    PARTITION BY profil
    ORDER BY COUNT(DISTINCT id_offre) DESC
  ) AS rang_dans_profil
FROM competences
WHERE competence != 'non_détecté'
GROUP BY profil, famille, competence
ORDER BY profil, rang_dans_profil;
```

- RANK() OVER (PARTITION BY profil) attribue un rang indépendant par profil : compétence n°1 du Data Engineer, du DevOps, etc.
- COUNT(DISTINCT id_offre) évite de compter plusieurs fois une compétence détectée dans plusieurs champs de la même offre.
- Le filtre competence != 'non_détecté' exclut les lignes de traçabilité NLP des agrégats.

Table Gold : salaires_par_profil.parquet

Objectif : produire des statistiques salariales complètes par combinaison profil × ville × type de contrat.

Colonnes produites : profil, ville, type_contrat, nb_offres, nb_offres_avec_salaire, salaire_median_mad, salaire_moyen_mad, salaire_q1_mad, salaire_q3_mad, salaire_min_observe, salaire_max_observe.

Requête DuckDB :

```
● ● ● DuckDB SQL – salaires_par_profil.parquet

SELECT
  profil_normalise AS profil,
  ville_std AS ville,
  type_contrat_std AS type_contrat,
  COUNT(*) AS nb_offres,
  COUNT(*) FILTER (WHERE salaire_connu) AS nb_offres_avec_salaire,
  ROUND(MEDIAN(salaire_median_mad)
        FILTER (WHERE salaire_connu), 0) AS salaire_median_mad,
  ROUND(AVG(salaire_median_mad)
        FILTER (WHERE salaire_connu), 0) AS salaire_moyen_mad,
  ROUND(PERCENTILE_CONT(0.25) WITHIN GROUP
        (ORDER BY salaire_median_mad)
        FILTER (WHERE salaire_connu), 0) AS salaire_q1_mad,
  ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP
        (ORDER BY salaire_median_mad)
        FILTER (WHERE salaire_connu), 0) AS salaire_q3_mad,
  ROUND(MIN(salaire_min_mad)
        FILTER (WHERE salaire_connu), 0) AS salaire_min_observe,
  ROUND(MAX(salaire_max_mad)
        FILTER (WHERE salaire_connu), 0) AS salaire_max_observe
FROM offres_clean
GROUP BY profil_normalise, ville_std, type_contrat_std
HAVING COUNT(*) >= 5
ORDER BY nb_offres DESC;
```

- supprime les cellules trop petites pour produire des statistiques robustes.
- FILTER (WHERE salaire_connu) garantit que seules les offres avec salaire déclaré entrent dans les calculs.
- PERCENTILE_CONT calcule les quartiles par interpolation continue, adaptée aux distributions salariales.

Table Gold : offres_par_ville.parquet

Objectif : mesurer le volume d’offres et la part de télétravail par ville, profil et mois afin de soutenir l’analyse géographique et temporelle.

Colonnes produites : ville, region_admin, profil, annee, mois, nb_offres, nb_offres_remote, pct_remote.

Requête DuckDB :


```

    DuckDB SQL – offres_par_ville.parquet

SELECT
  ville_std AS ville,
  region_admin,
  profil_normalise AS profil,
  annee,
  mois,
  COUNT(*) AS nb_offres,
  COUNT(*) FILTER (
    WHERE teletravail ILIKE '%télétravail%'
      OR teletravail ILIKE '%remote%'
      OR teletravail ILIKE '%hybride%'
  ) AS nb_offres_remote,
  ROUND(
    COUNT(*) FILTER (
      WHERE teletravail ILIKE '%télétravail%'
        OR teletravail ILIKE '%remote%'
        OR teletravail ILIKE '%hybride%'
    ) * 100.0 / NULLIF(COUNT(*), 0),
    1
  ) AS pct_remote
FROM offres_clean
GROUP BY ville_std, region_admin, profil_normalise, annee, mois
ORDER BY nb_offres DESC;

```

- ILIKE capture les variantes de saisie indépendamment de la casse : Remote, REMOTE, remote.
- NULLIF(COUNT(*), 0) protège le calcul du pourcentage contre une division par zéro.
- La granularité ville × profil × mois alimente directement la comparaison Tanger / Casablanca / Rabat dans le notebook.

Table Gold : entreprises_recruteurs.parquet

Objectif : identifier les entreprises les plus actives sur le marché IT marocain, la diversité des profils recherchés et le niveau de rémunération moyen proposé.

Colonnes produites : entreprise, ville, nb_offres_publiees, nb_profils_différents, salaire_moyen_propose, profils_recrutes, premiere_offre, derniere_offre.

Requête DuckDB :

```

    DuckDB SQL – entreprises_recruteurs.parquet

SELECT
  entreprise,
  ville_std AS ville,
  COUNT(*) AS nb_offres_publiees,
  COUNT(DISTINCT profil_normalise) AS nb_profils_différents,
  ROUND(AVG(salaire_median_mad)
    FILTER (WHERE salaire_connu), 0) AS salaire_moyen_propose,
  ARRAY_AGG(DISTINCT profil_normalise
    ORDER BY profil_normalise) AS profils_recrutes,
  MIN(date_publication_std) AS premiere_offre,
  MAX(date_publication_std) AS derniere_offre
FROM offres_clean
WHERE entreprise IS NOT NULL AND entreprise != ''
GROUP BY entreprise, ville_std
HAVING COUNT(*) >= 3
ORDER BY nb_offres_publiees DESC
LIMIT 100;

```

- ARRAY_AGG(DISTINCT profil_normalise) retourne la liste des profils recrutés par entreprise, exploitable dans le notebook.
- HAVING COUNT(*) >= 3 filtre les recruteurs anecdotiques pour conserver les entreprises réellement actives.
- MIN / MAX(date_publication_std) délimitent la fenêtre d'activité de chaque recruteur sur la période 2023-2024.
- Le LIMIT 100 conserve les principaux recruteurs par volume, ce qui suffit pour l'analyse stratégique.

Table Gold : tendances_mensuelles.parquet

Objectif : suivre l'évolution mensuelle du volume d'offres et des salaires par profil, puis calculer le taux de variation mois sur mois.

Colonnes produites : annee, mois, profil, nb_offres, salaire_moyen_mois, nb_offres_mois_precedent, evolution_pct.

Requête DuckDB :

```
DuckDB SQL – tendances_mensuelles.parquet

SELECT
  annee,
  mois,
  profil_normalise AS profil,
  COUNT(*) AS nb_offres,
  ROUND(AVG(salaire_median_mad)
        FILTER (WHERE salaire_connu), 0) AS salaire_moyen_mois,
  LAG(COUNT(*)) OVER (
    PARTITION BY profil_normalise
    ORDER BY annee, mois
  ) AS nb_offres_mois_precedent,
  ROUND(
    (COUNT(*) - LAG(COUNT(*)) OVER (
      PARTITION BY profil_normalise ORDER BY annee, mois
    )) * 100.0 / NULLIF(
      LAG(COUNT(*)) OVER (
        PARTITION BY profil_normalise ORDER BY annee, mois
      ), 0
    ), 1
  ) AS evolution_pct
FROM offres_clean
WHERE annee IS NOT NULL AND mois IS NOT NULL
GROUP BY annee, mois, profil_normalise
ORDER BY profil_normalise, annee, mois;
```

- LAG() récupère la valeur du mois précédent pour chaque profil, sans jointure auto-référentielle.
- NULLIF(..., 0) évite une division par zéro pour le premier mois de chaque profil.
- Le tri par annee, mois garantit un ordre chronologique stable même si DuckDB lit les partitions dans un ordre différent

Vérification post-construction

Après la génération des fichiers Gold, la fonction `verifier_gold()` confirme que les cinq tables sont lisibles et affiche leur schéma. Cette vérification permet de valider rapidement que les fichiers Parquet produits sont exploitables par DuckDB, pandas ou un outil BI.

Table 12 – Contrôles de validation des tables Gold

Table	Contrôle attendu
top_competences	fichier lisible, 6 colonnes : profil, famille, competence, nb_offres_mentionnent, pct_offres_total, rang_dans_profil
salaires_par_profil	fichier lisible, 11 colonnes : statistiques salariales par profil, ville et contrat
offres_par_ville	fichier lisible, 8 colonnes : volume, remote et granularité ville/profil/mois
entreprises_recruteurs	fichier lisible, 8 colonnes : activité des recruteurs, profils et période d'activité
tendances_mensuelles	fichier lisible, 7 colonnes : volume mensuel, salaire moyen et évolution

Règles de gouvernance Gold

- Les tables Gold sont régénérées entièrement à chaque exécution du pipeline : une nouvelle exécution de `main.py` remplace les fichiers existants.
- Aucun processus aval ne modifie ces fichiers. Le notebook, les dashboards et les rapports les consomment en lecture seule via `read_parquet()` ou DuckDB.
- Le format Parquet Snappy garantit la portabilité : les tables sont requêttables directement par DuckDB, pandas, Metabase ou Power BI sans serveur intermédiaire.

Synthèse

L'agrégation Gold transforme le Data Lake en couche décisionnelle. Les données Silver détaillées restent disponibles pour l'audit, tandis que les tables Gold fournissent des indicateurs directement exploitables : compétences prioritaires, salaires par segment, répartition géographique, recruteurs actifs et tendances temporelles.

Code source du projet : l'ensemble des scripts du pipeline, du notebook d'analyse et des livrables associés est disponible sur le dépôt GitHub suivant : [GitHub](#)